

Engine Selection: Apache Flink vs. Apache Spark Structured Streaming

Which engine fits Bus Bunching Detection, which fits Ward Energy Aggregation, and why — assessed across state size, latency requirement, RTO and operational complexity

COURSE CODE	TARGET CITY	DELIVERABLE	COMPONENT
DSE ZG556 / CC ZG556	MetroConnect (Pop. 4.2M)	Task C — Processing Layer	Engine Selection Comparison

1 Engine Assignment Across the Implemented Jobs

The processing layer is split across two frameworks because its workloads split cleanly into two shapes. Incident detection needs state that persists across events and a decision emitted the moment a temporal condition is met. Analytics reduces to a window, an aggregate and a join — a query, not an operator. Each job is assigned to the engine whose execution model already matches it, so neither pipeline is over-engineered.

IMPLEMENTED JOB	LATENCY TARGET	ENGINE	DECIDING FACTOR
AQI emergency — alert when AQI > 300	Within 2 min	FLINK	Threshold breach must fire on arrival of the offending reading, not at the next batch boundary
Traffic gridlock — avg wait > 180 s over 3 consecutive cycles	Sub-2-minute	FLINK	Per-junction state carried across a consecutive-cycle sequence — a running condition, not a window
Bus bunching — two buses on a route within 200 m for > 5 min	Sub-2-minute	FLINK	Event-time timers plus fine-grained keyed geospatial state — see Section 2
Ward energy aggregation — 15-min tumbling window per <code>ward_id</code>	15-min windows	SPARK	Declarative windowed SQL, watermark-bounded state, trivial dual sink to Kafka and Parquet — see Section 2
Rolling AQI advisory — 10-min rolling avg per zone, joined to static zone profile	10-min windows	SPARK	Stream-to-static join and rolling aggregation expressed directly in SQL with update output mode

2 Dimension-by-Dimension Comparison

DIMENSION	BUS BUNCHING DETECTION → APACHE FLINK	WARD ENERGY AGGREGATION → APACHE SPARK
Latency Requirement	Flink is a true event-at-a-time engine with sub-millisecond to low-millisecond latency, evaluating the condition when the event arrives or when an event-time timer fires. Spark's micro-batching imposes a delay floor set by the trigger interval — spent latency buying nothing on a strict sub-2-minute SLA. The timer semantics matter as much as raw speed: "within 200 m for five minutes" is a registered event-time timer, and awkward as a repeated batch query.	The output unit is a 15-minute window, so a few seconds of micro-batch delay is imperceptible to any downstream dashboard or report. Spark's latency floor simply does not bind here, and continuous per-event execution would add overhead for no gain.
State Size & Management	Requires holding continuous geospatial positions for thousands of buses and evaluating pairwise distances constantly. Flink's fine-grained mutable <code>KeyedState</code> (<code>ValueState</code> , <code>MapState</code>) on RocksDB is read and updated per key; keying by <code>route_id</code> touches only the affected route rather than shuffling a large <code>DataFrame</code> each micro-batch. State is long-lived and irregularly shaped.	State is partial aggregates — sums, averages, maxima — held only while the window is open. Spark is optimised for SQL group-by aggregation and handles late data via watermarking; closed windows drop their state automatically, so state stays strictly bounded with no manual TTL. The opposite profile: short-lived, uniform, self-clearing.
Recovery Time Objective	Emergency detection tolerates little downtime. Flink's asynchronous distributed checkpointing (Chandy-Lamport) snapshots state without pausing the pipeline; after a failure the job resumes from the last checkpoint with exactly-once state guarantees, giving an RTO in seconds. Durability costs nothing in steady-state latency.	A reporting pipeline has a materially more relaxed RTO. Spark uses write-ahead logs and state store checkpoints, recomputing a lost micro-batch from lineage. Recovery is slower than snapshot restore, but a delayed 15-minute rollup has no operational consequence — and accepting a worse RTO where it is free is what makes the strong RTO on the alert path affordable.
Operational Complexity	The dimension where Flink loses, accepted deliberately. Custom <code>KeyedProcessFunction</code> implementations with registered timers demand deeper expertise than Spark SQL, and RocksDB backends need tuning. Justified because no declarative alternative expresses geospatial-plus-temporal logic without contortion — the complexity buys a capability, not just speed.	Where Spark wins decisively. A unified declarative <code>DataFrame</code> /SQL API makes the aggregation a windowed group-by rather than a custom operator, and the dual sink — Kafka for dashboards, Parquet partitioned by <code>ward_id</code> and date — is straightforward via <code>foreachBatch</code> , where an equivalent Flink file sink is verbose and needs tuning. It also stays maintainable in SQL after handover.

Conclusion

Flink handles the three incident detectors; Spark handles ward energy aggregation and the rolling AQI advisory. The split follows the shape of the workloads rather than a preference for either engine — Flink only where per-event timers and fine-grained mutable state are genuinely required, Spark everywhere the problem reduces to a windowed query. The alerting path is not slowed by batch semantics it cannot afford, and the reporting path carries no operational complexity it does not need.